

COMP4350 – MIDTERM PROJECT REPORT

Ayfernaz Baygın - 22070006068

PART A – MLP REGRESSION

1. Problem Description

In this part of the project, a Multi-Layer Perceptron (MLP) neural network was developed to predict house prices using the given dataset. The dataset consists of three input features and one target variable (price). The model was implemented from scratch using Python without using any machine learning libraries, as required by the assignment.

2. Data Preprocessing

Before training the neural network, all variables were normalized using Min-Max normalization as required.

The dataset includes a categorical feature named **Neighborhood**. Since neural networks require numerical input, one-hot encoding was applied to this feature. As a result, the number of input features increased from 3 to 6 after preprocessing.

The test dataset was encoded using the same categories obtained from the training dataset and normalized using the minimum and maximum values calculated from the training set.

3. Model Architecture

The MLP model consists of:

- Input layer: 6 neurons
- Hidden layers: 1
- Hidden layer neurons: 16
- Output layer: 1 neuron

Activation functions:

- Hidden layer: tanh
- Output layer: linear

4. Training Configuration

The model was trained using backpropagation and gradient descent with momentum.

- Learning rate: 0.01
- Momentum: 0.9
- Maximum number of epochs: 8000

Stopping criteria:

- Maximum epoch limit (8000)
- Early stopping based on loss improvement (tolerance = $1e-10$, patience = 800)

The training process reached the maximum epoch limit, indicating that early stopping was not triggered before the final iteration.

5. Results

Training Results:

- MAE: 2700746.015079
- MSE: 17259841246655.916016
- RMSE: 4154496.509405
- R^2 : 0.314390

Test Results (Macro Average):

- MAE: 3144322.403733
- MSE: 24774707104490.753906
- RMSE: 4977419.723561
- R^2 : 0.245548

6. Discussion

The model was able to learn part of the relationship between the input features and house prices. However, the relatively low R^2 values indicate that the predictive performance remains moderate.

This may be due to the limited number of features or the complexity of the relationship between input variables and the target variable. Further improvements could be achieved by tuning hyperparameters or increasing model complexity.

The final weights and biases of the MLP model are provided in the report.txt file.

PART B – K-MEANS CLUSTERING

1. Problem Description

In this part, K-Means clustering was implemented from scratch to group the dataset into clusters. The dataset contains three numerical features and no labels.

2. Data Preprocessing

All features were normalized using Min-Max normalization before applying the clustering algorithm.

3. Methodology

The K-Means algorithm was implemented using NumPy without using any built-in clustering libraries.

Steps:

1. Randomly initialize cluster centers
2. Assign each data point to the nearest cluster
3. Update cluster centers
4. Repeat until convergence

The algorithm converged in 10 iterations. The same results were obtained in repeated runs, ensuring reproducibility of the clustering process.

4. Results

Cluster distribution:

- Cluster 1: 73 records
- Cluster 2: 82 records
- Cluster 3: 45 records

Evaluation metrics:

- WCSS: 5.420842
- BCSS: 29.669574
- Dunn Index: 0.066154

5. Visualization

Cluster visualization was performed using different feature pairs:

- Başçevre vs Boy
- Boy vs Kilo

The clusters are clearly separable and follow a consistent structure across different projections.

6. Discussion

The clustering results show that the dataset naturally forms three groups. The separation between clusters is relatively clear, especially in the visualizations. The evaluation metrics support that the clustering structure is meaningful.

GENERAL CONCLUSION

In this project, both supervised (MLP regression) and unsupervised (K-Means clustering) learning methods were implemented from scratch.

The MLP model provided moderate prediction performance, while the K-Means algorithm successfully identified meaningful clusters in the dataset.

All implementations were completed using only NumPy and Pandas, in accordance with the assignment requirements.